

RELATÓRIO DE BENCHMARK

Avaliação Somativa de Matemática para computação

Professor: Rodrigo Attique

Alunos: Ana Carolina Gonzalis, Cecilio Miguel Neto, Haroldo Faria e José Carlos Fiaschi Jr.

Análise comparativa de desempenho computacional utilizando algoritmos matemáticos e consulta SQL em Python.

1. Introdução

O presente trabalho tem como objetivo analisar o desempenho de diferentes máquinas na execução de algoritmos matemáticos e operações de banco de dados utilizando a linguagem Python. Foram realizados testes de benchmark envolvendo estruturas iterativas, recursivas e consultas SQL.

```
=====
RELATÓRIO DE BENCHMARK - GREEN SOFTWARE ENGINEERING
=====
```

```
SO: Windows 11 (AMD64)
```

```
Processador: Intel64 Family 6 Model 140 Stepping 1, GenuineIntel
```

```
Linguagem: Python 3.14.3
=====
```

```
--> Testando algoritmos matemáticos com N = 20
```

```
[Fibonacci (Loop)          ] Execuções: 100 | Média de Tempo: 0.00000130 segundos
[Fibonacci (Recursão)      ] Execuções: 10  | Média de Tempo: 0.00152339 segundos
[Fatorial (Loop)           ] Execuções: 100 | Média de Tempo: 0.00000180 segundos
[Fatorial (Recursão)       ] Execuções: 100 | Média de Tempo: 0.00000200 segundos
[Somatória (Loop)          ] Execuções: 100 | Média de Tempo: 0.00000078 segundos
[Somatória (Recursão)      ] Execuções: 100 | Média de Tempo: 0.00000169 segundos
```

```
--> Testando carga de Banco de Dados (1000 registros)
```

```
[Consulta SQL (SELECT)     ] Execuções: 100 | Média de Tempo: 0.00067453 segundos
=====
```

```
PS E:\MEUS DOCUMENTOS\aula_python>
```

Maquina Haroldo

```
C:\Users\junio>C:/Users/junio/AppData/Local/Python/pythoncore-3.14-64/python.exe c
```

```
=====
RELATÓRIO DE BENCHMARK - GREEN SOFTWARE ENGINEERING
=====
```

```
SO: Windows 11 (AMD64)
```

```
Processador: Intel64 Family 6 Model 186 Stepping 2, GenuineIntel
```

```
Linguagem: Python 3.14.3
=====
```

```
--> Testando algoritmos matemáticos com N = 20
```

```
[Fibonacci (Loop)          ] Execuções: 100 | Média de Tempo: 0.00000053 segundos
[Fibonacci (Recursão)      ] Execuções: 10  | Média de Tempo: 0.00076455 segundos
[Fatorial (Loop)           ] Execuções: 100 | Média de Tempo: 0.00000057 segundos
[Fatorial (Recursão)       ] Execuções: 100 | Média de Tempo: 0.00000079 segundos
[Somatória (Loop)          ] Execuções: 100 | Média de Tempo: 0.00000036 segundos
[Somatória (Recursão)      ] Execuções: 100 | Média de Tempo: 0.00000071 segundos
```

```
--> Testando carga de Banco de Dados (1000 registros)
```

```
[Consulta SQL (SELECT)     ] Execuções: 100 | Média de Tempo: 0.00038187 segundos
=====
```

Maquina Junio (Jose Carlos Fiaschi Junior)

RELATÓRIO DE BENCHMARK - GREEN SOFTWARE ENGINEERING

SO: Windows 11 (AMD64)

Processador: Intel64 Family 6 Model 186 Stepping 2, GenuineIntel

Linguagem: Python 3.14.3

--> Testando algoritmos matemáticos com N = 20

[Fibonacci (Loop)]	] Execuções: 100 Média de Tempo: 0.00000059 segundos
[Fibonacci (Recursão)]	] Execuções: 10 Média de Tempo: 0.00074757 segundos
[Fatorial (Loop)]	] Execuções: 100 Média de Tempo: 0.00000063 segundos
[Fatorial (Recursão)]	] Execuções: 100 Média de Tempo: 0.00000163 segundos
[Somatória (Loop)]	] Execuções: 100 Média de Tempo: 0.00000056 segundos
[Somatória (Recursão)]	] Execuções: 100 Média de Tempo: 0.00000100 segundos

--> Testando carga de Banco de Dados (1000 registros)

[Consulta SQL (SELECT)]	] Execuções: 100 Média de Tempo: 0.00042569 segundos
-------------------------	--

PS C:\Users\Usuario\Desktop\AVALIAÇÃO ATTIQUE>

Maquina Neto (Cecilio Miguel Neto)

```
PS C:\Users\carol\OneDrive\Desktop\FATEC - 2º MODULO\Matematica\Avaliacao Somativa> &
grams\Python\Python314\python.exe "c:/Users/carol/OneDrive/Desktop/FATEC - 2º MODULO/
liação_somativa_mat_2sem (1).py"

=====
RELATÓRIO DE BENCHMARK - GREEN SOFTWARE ENGINEERING
=====

SO: Windows 11 (AMD64)
Processador: Intel64 Family 6 Model 140 Stepping 1, GenuineIntel
Linguagem: Python 3.14.3
=====

--> Testando algoritmos matemáticos com N = 20

[Fibonacci (Loop)          ] Execuções: 100 | Média de Tempo: 0.00000115 segundos
[Fibonacci (Recursão)      ] Execuções: 10 | Média de Tempo: 0.00172607 segundos
[Fatorial (Loop)           ] Execuções: 100 | Média de Tempo: 0.00000137 segundos
[Fatorial (Recursão)       ] Execuções: 100 | Média de Tempo: 0.00000211 segundos
[Somatória (Loop)          ] Execuções: 100 | Média de Tempo: 0.00000088 segundos
[Somatória (Recursão)      ] Execuções: 100 | Média de Tempo: 0.00000190 segundos

--> Testando carga de Banco de Dados (1000 registros)
[Consulta SQL (SELECT)     ] Execuções: 100 | Média de Tempo: 0.00085586 segundos
=====
PS C:\Users\carol\OneDrive\Desktop\FATEC - 2º MODULO\Matematica\Avaliacao Somativa>
```

Maquina carol (Ana Carolina Gonzalis)

2. Metodologia

Os algoritmos foram executados múltiplas vezes em quatro computadores distintos utilizando Python 3.14.3. Foram avaliados os algoritmos de Fibonacci, Fatorial, Somatória e uma consulta SQL com aproximadamente 1000 registros.

3. Configuração das Máquinas

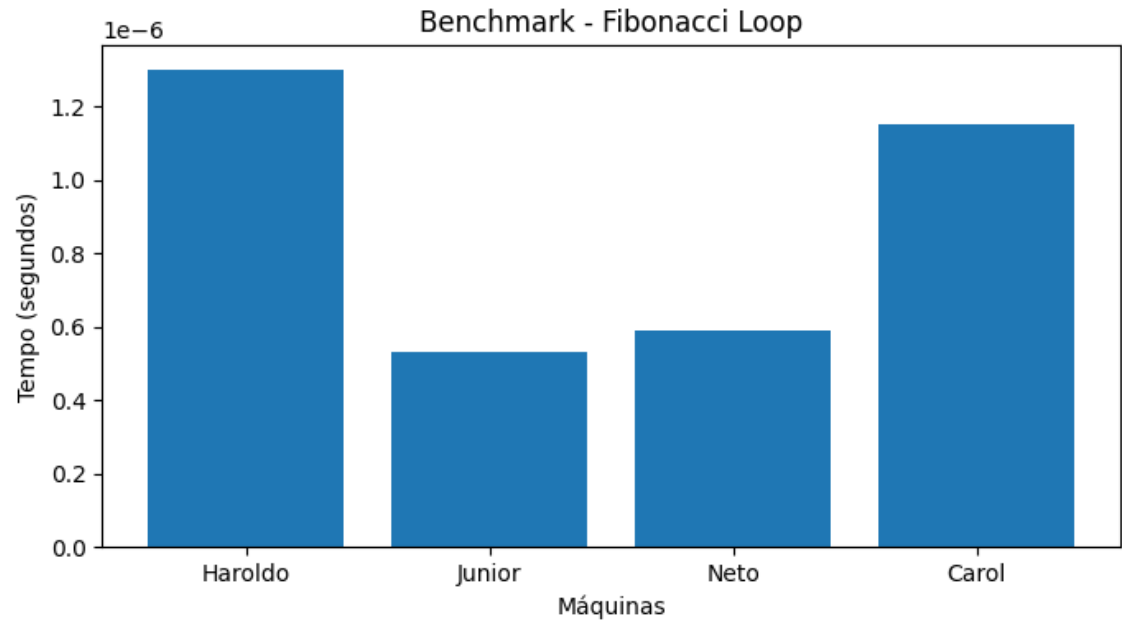
Usuário	Processador	Mem. RAM	Sistema Operacional	Arquitetura
Haroldo	Intel i7 Family 6 Model 140	8gb	Windows 11	AMD64
Junior	Intel i7 Family 6 Model 186	16gb	Windows 11	AMD64
Neto	Intel i5 Family 6 Model 186	8gb	Windows 11	AMD64

Carol	Intel i3 Family 6 Model 140	20gb	Windows 11	AMD64
-------	--------------------------------	------	------------	-------

4. Resultados Consolidados

Máquina	Fibonacci Loop	Fibonacci Recursão	Fatorial Loop	Consulta SQL
Haroldo	0.00000130	0.00152339	0.00000180	0.00067453
Junior	0.00000053	0.00076455	0.00000057	0.00038187
Neto	0.00000059	0.00074757	0.00000063	0.00042569
Carol	0.00000115	0.00172607	0.00000137	0.00085586

5. Gráficos Comparativos



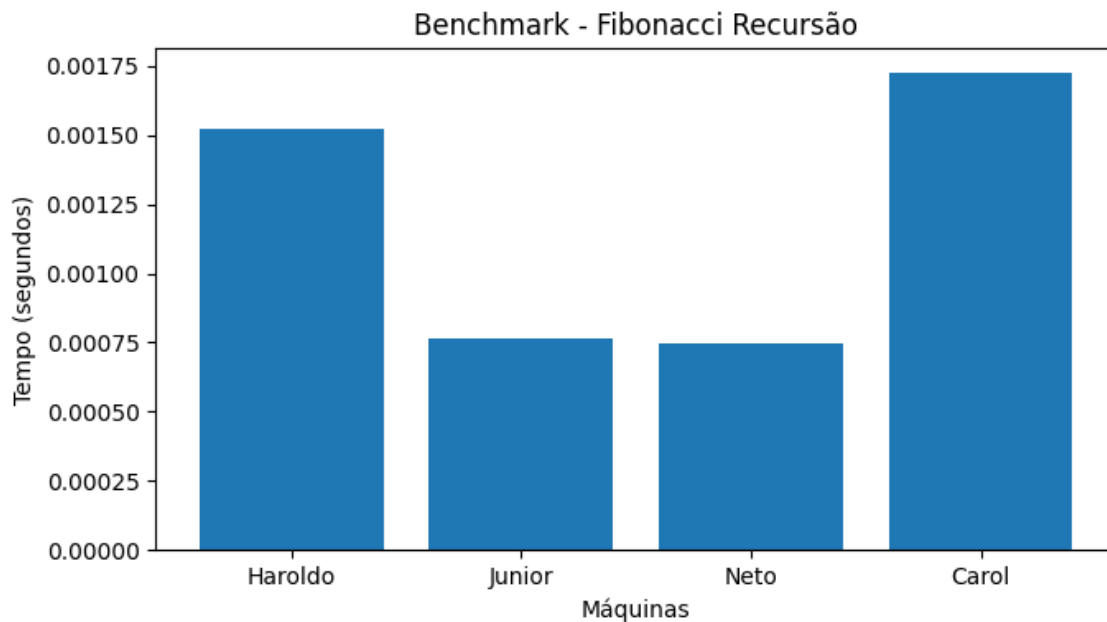
Este gráfico compara o **tempo médio de execução do algoritmo Fibonacci em loop** nas quatro máquinas avaliadas. O eixo vertical mostra o tempo em segundos, porém em escala científica 1e-6, ou seja, os valores estão na casa dos **microssegundos**. Quanto menor a barra, melhor o desempenho.

O melhor resultado foi da máquina **Junior**, com aproximadamente **0,00000053 s**, seguida por **Neto**, com **0,00000059 s**. Isso indica que essas duas máquinas executaram o Fibonacci em loop de forma mais eficiente. Na prática, Junior foi cerca de **59% mais rápido que Haroldo** e aproximadamente **54% mais rápido que Carol** nesse teste específico.

A máquina **Haroldo** apresentou o maior tempo médio, cerca de **0,00000130 s**, ficando com o desempenho mais lento neste algoritmo. A máquina **Carol** ficou próxima, com aproximadamente **0,00000115 s**. Mesmo assim, é importante destacar que todos os tempos são extremamente baixos, pois o Fibonacci em loop é um algoritmo leve, com baixa exigência computacional.

Tecnicamente, o resultado confirma que a implementação em **loop** tem excelente eficiência, pois evita chamadas recursivas e utiliza apenas atualização simples de variáveis. As diferenças entre as máquinas podem estar relacionadas ao modelo do processador, frequência de operação, quantidade de processos abertos no momento do teste, consumo de memória, modo de energia do Windows e condições gerais do sistema.

Conclusão direta: **Junior e Neto tiveram melhor desempenho no Fibonacci em loop**, enquanto **Haroldo e Carol apresentaram tempos maiores**. Porém, como os valores estão em microssegundos, a diferença é pequena em uso prático, mas importante para fins de benchmark comparativo.



Este gráfico mostra o **tempo médio de execução do algoritmo Fibonacci por recursão** em quatro máquinas: Haroldo, Junior, Neto e Carol. O eixo vertical indica o tempo em segundos; quanto menor a barra, melhor o desempenho.

A leitura dos resultados é:

Máquina	Tempo médio aproximado
Neto	0,000747 s
Junior	0,000765 s
Haroldo	0,001523 s
Carol	0,001726 s

A máquina **Neto** teve o melhor desempenho, com a menor barra. Logo depois vem **Junior**, praticamente empatado. Já **Haroldo** e **Carol** apresentaram tempos bem maiores, sendo **Carol a máquina mais lenta** nesse teste.

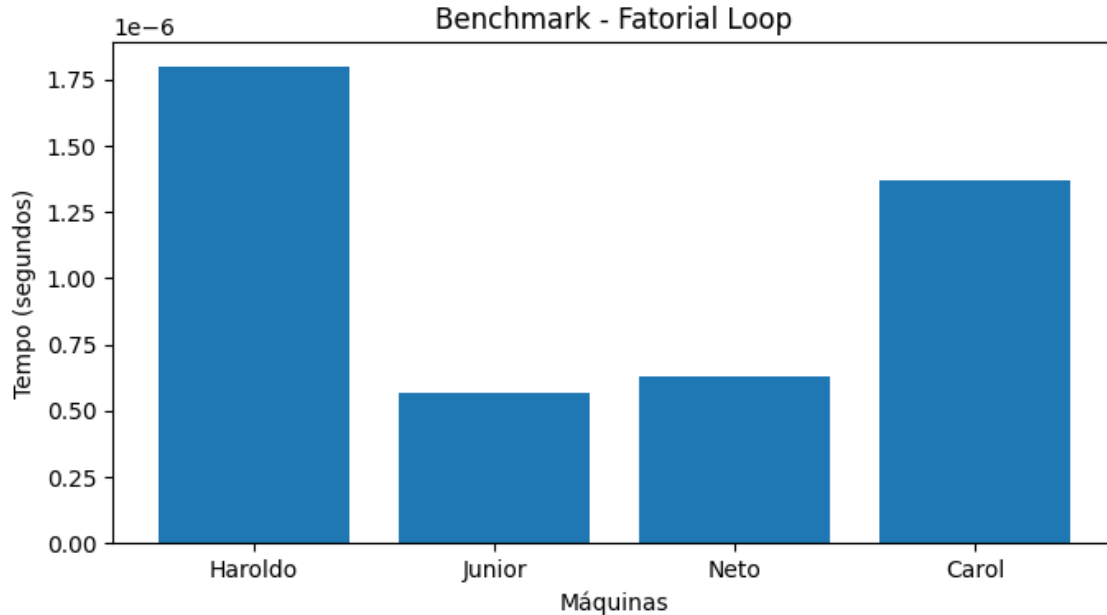
A diferença é importante porque o Fibonacci recursivo é um algoritmo pesado. Ele chama a própria função várias vezes, repetindo cálculos já feitos. Por isso, exige mais do processador e evidencia melhor a diferença entre as máquinas.

Comparando os extremos, **Neto foi cerca de 57% mais rápido que Carol**. Isso significa que, nesse tipo de processamento recursivo, a máquina da Carol levou mais que o dobro do tempo da máquina do Neto.

Tecnicamente, o gráfico mostra três conclusões principais:

1. **Junior e Neto tiveram melhor desempenho computacional.**
2. **Haroldo e Carol tiveram maior tempo de processamento.**
3. **A recursão é menos eficiente que o loop**, porque gera muitas chamadas internas da função.

Conclusão direta: o gráfico comprova que algoritmos recursivos consomem mais processamento, e que diferenças de hardware ficam mais visíveis quando o algoritmo exige mais da máquina.



Este gráfico apresenta o **tempo médio de execução do algoritmo Fatorial utilizando loop** nas quatro máquinas avaliadas. O eixo vertical está em segundos,

com escala $1e-6$, ou seja, os resultados estão em **microsegundos**. Quanto menor a barra, melhor o desempenho.

Os tempos aproximados foram:

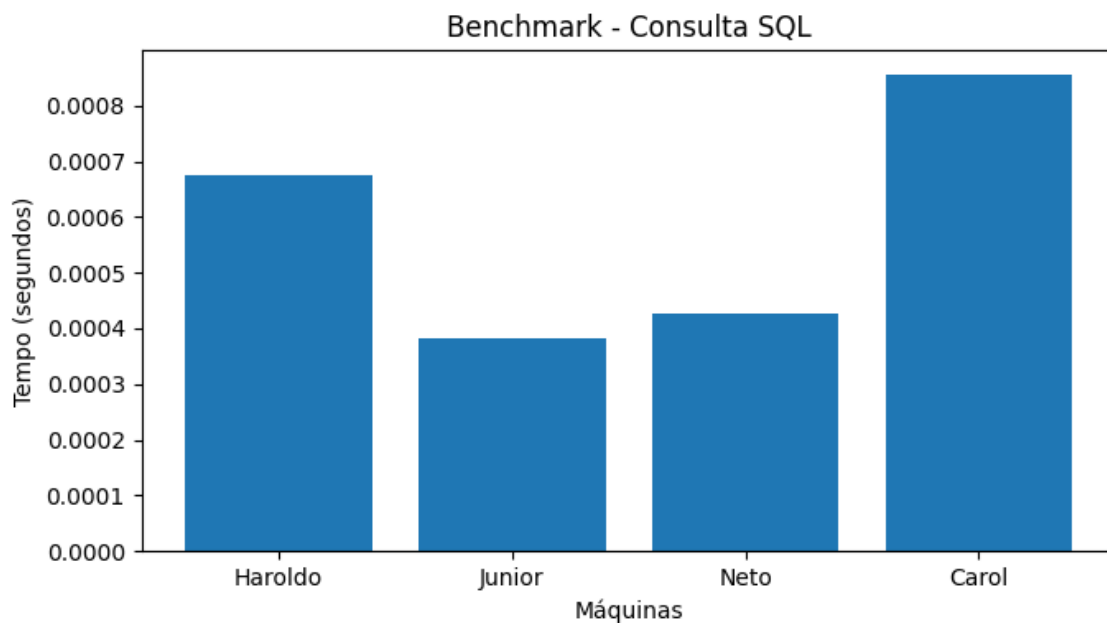
Máquina	Tempo médio aproximado
Junior	0,00000057 s
Neto	0,00000063 s
Carol	0,00000137 s
Haroldo	0,00000180 s

A máquina **Junior** apresentou o melhor desempenho, seguida de perto por **Neto**. Isso indica que essas duas máquinas executaram o cálculo fatorial por loop com maior eficiência. Já **Carol** teve desempenho intermediário, enquanto **Haroldo** apresentou o maior tempo médio, sendo a máquina mais lenta nesse teste.

Tecnicamente, o fatorial em loop é um algoritmo simples e eficiente. Ele realiza multiplicações sucessivas dentro de uma estrutura for, sem chamadas recursivas. Por isso, o custo computacional é baixo e previsível. Mesmo assim, o gráfico mostra diferença entre as máquinas, provavelmente relacionada ao processador, gerenciamento de energia, processos em segundo plano e estado geral do sistema no momento da execução.

A comparação mais relevante é entre **Junior** e **Haroldo**. Junior executou o algoritmo em cerca de **0,00000057 s**, enquanto Haroldo levou aproximadamente **0,00000180 s**. Isso significa que Junior foi cerca de **68% mais rápido** nesse teste.

Conclusão direta: **Junior e Neto tiveram melhor desempenho no Fatorial Loop**, enquanto **Haroldo apresentou o pior tempo médio**. Como os valores são muito pequenos, a diferença prática no uso comum é quase imperceptível, mas para benchmark ela demonstra claramente qual máquina respondeu melhor em operações matemáticas simples.



Este gráfico apresenta o desempenho das quatro máquinas na **consulta SQL com aproximadamente 1000 registros**. O eixo vertical mostra o tempo médio em segundos; quanto menor a barra, melhor foi o desempenho da máquina na leitura dos dados.

Os tempos médios aproximados foram:

Máquina	Tempo médio
Junior	0,00038187 s
Neto	0,00042569 s
Haroldo	0,00067453 s
Carol	0,00085586 s

A máquina **Junior** apresentou o melhor resultado, com o menor tempo de execução. Logo depois vem **Neto**, com desempenho muito próximo. Isso indica que essas duas máquinas foram mais eficientes na execução da consulta SQL.

A máquina **Haroldo** ficou em terceiro lugar, com tempo médio maior que Junior e Neto. Já a máquina **Carol** apresentou o maior tempo de execução, sendo a mais lenta nesse teste específico.

Tecnicamente, esse gráfico mede mais do que apenas cálculo matemático. A consulta SQL envolve:

- criação/acesso ao banco de dados;
- leitura de registros;
- busca dos dados na tabela;
- transferência dos dados para a memória;
- processamento interno do SQLite/Python.

Por isso, esse teste depende não apenas do processador, mas também da memória RAM, velocidade do disco, quantidade de processos abertos e desempenho geral do sistema operacional.

Comparando os extremos, **Junior foi aproximadamente 55% mais rápido que Carol** na consulta SQL. Isso mostra que, em operações com dados, pequenas diferenças de hardware e estado da máquina podem impactar bastante o tempo de resposta.

Conclusão direta: **Junior teve o melhor desempenho na consulta SQL**, seguido por **Neto**. **Carol apresentou o maior tempo médio**, indicando menor eficiência na leitura dos 1000 registros durante o benchmark.

6. Análise Técnica

Os resultados demonstram que os algoritmos implementados em loop apresentaram desempenho superior quando comparados às versões recursivas. O Fibonacci recursivo apresentou maior tempo de execução devido ao elevado número

de chamadas recursivas. As diferenças entre os tempos médios também evidenciam a influência do hardware no desempenho computacional.

7. Conclusão

Conclui-se que estruturas iterativas possuem maior eficiência computacional em relação às estruturas recursivas para os algoritmos analisados. O benchmark permitiu comparar o desempenho entre diferentes máquinas e demonstrar a importância da análise de desempenho em sistemas computacionais.